AI-Powered Outfit Recommendation Platform

1. Project Objective

The goal of this project is to build a simplified AI-powered product recommendation system using FastAPI. Learners will begin with a traditional E-commerce backend and enhance it into an AI-Commerce system. Users provide natural language prompts (e.g., 'Suggest the best outfit to wear to temple'), and the system returns relevant outfit recommendations along with product images using semantic search techniques.

2. Learning Outcomes

• Understand traditional E-commerce product listing systems

• Design and build structured FastAPI backend applications

• Implement semantic search using embeddings

• Apply cosine similarity ranking

• Return contextual recommendations with images and metadata

• Track model versions and log AI usage

3. System Architecture Overview

User Prompt → FastAPI → Query Embedding → Cosine Similarity Ranking → Top Outfit Recommendations (with Images)

4. Mandatory Project Structure

```
ai-product-search/
└── FastApi/
    ├── README.md
    ├── CODE_STRUCTURE.md
    ├── DESIGN_DECISIONS.md
    ├── API_DOCUMENTATION.md
    ├── requirements.txt
```

```
└── app/
    ├── main.py
    ├── __init__.py
    ├── static/
    │   ├── css/style.css
    │   ├── images/(product images)
    │   └── js/cart.js
    └── templates/(base.html, cart.html, index.html)
```

5. API Requirements

Product APIs:

• GET /products

• GET /products/{id}

• POST /products

AI Recommendation API:

• POST /products/recommend

• Request Body: { 'prompt': 'User natural language query' }

• Response must include query text, model version, top 5 products, image URLs, similarity score

6. Semantic Search Implementation

6.1 Embedding Generation (Indexing Phase)

- Generate vector embeddings for each product using the selected embedding model.
- Ensure embedding dimensions are consistent across all records.
- Store product embeddings in a Vector Database or vector storage module.
- Attach relevant metadata to each vector (product ID, category, image URL, price, etc.).
- Build a vector index to enable efficient nearest-neighbor retrieval.

6.2 Query Intake and Preprocessing (Online Phase)

- Receive the user search query.
- Clean and normalize the input text.
- Convert the query into an embedding using the same embedding model used during indexing.

6.3 Similarity Search

- Perform nearest-neighbor search in the Vector Database.
- Compute similarity scores (e.g., cosine similarity) between the query vector and stored product vectors.

6.4 Ranking and Business Filtering

- Rank retrieved products based on similarity score.
- Apply additional business filters such as availability, category constraints, or pricing rules.

6.5 Response Construction

- Select the top N results (e.g., top 5).
- Retrieve associated metadata and image URLs.
- Format the final structured response for frontend delivery.

7. Logging & Model Version Tracking

Logging and Observability

- Log user query
- Log embedding model version
- Log latency and similarity scores
- Store search history for analytics, monitoring, and future improvements

8. Optional Advanced Enhancements (Good to Do)

• Intent extraction using LLM (ocaasion, season,style)

• Hybrid keyword + semantic search

• MongoDB integration for products

- MongoDB Atlas vector search
- Microservices architecture separation
- LLM lifecycle monitoring and controlled redeployment